

F2: $\tan(x)$

SOEN 6011 — Software Engineering Processes
Deliverable 3

Konan Brandon
Student ID: 40373534

Summer 2026

Contents

1	Introduction	2
2	Problem 7: Styling, Debugging, Static Analysis, Semantic Versioning, UI Design, and Accessibility	3
2.1	PEP 8 conformance and Flake8	3
2.2	Debugger: pdb	3
2.3	Static code analyser: Pylint	4
2.4	Semantic Versioning	5
2.5	User interface design principles	5
2.6	Accessibility	6
2.7	Use of generative artificial intelligence	6
3	Problem 8: Unit Testing (PyUnit)	9
3.1	Test suite and requirement traceability	9
3.2	Mutation testing	9
3.3	Use of generative artificial intelligence	9
4	Conclusion	11

1 Introduction

This deliverable modifies the from-scratch implementation of Deliverable 2 to satisfy Problem 7 and Problem 8: an established Python style, a styling tool, a debugger, a static analyser, Semantic Versioning, user interface design principles chosen by a mind map, accessibility, and a PyUnit test suite. The presentation medium for this deliverable is an A0 poster (**poster.tex**) rather than slides, but the artefacts, evidence, and reasoning behind every claim on that poster are recorded here in full, in the same manner as the Deliverable 2 report.

Four real defects were found while doing this work, none of them by reading code: a silent precision loss at large arguments, a signature mismatch between two modules, a documented command that failed on a fresh clone, and an unreachable validation branch masked by a later guard. That pattern repeats in this deliverable too, and is recorded where it occurred rather than smoothed over.

2 Problem 7: Styling, Debugging, Static Analysis, Semantic Versioning, UI Design, and Accessibility

2.1 PEP 8 conformance and Flake8

The five original modules plus the two added for this deliverable — `test_tan_math.py`, `pdb_demo.py`, and `check_contrast.py` — are checked together with Flake8 at the 79-column line length:

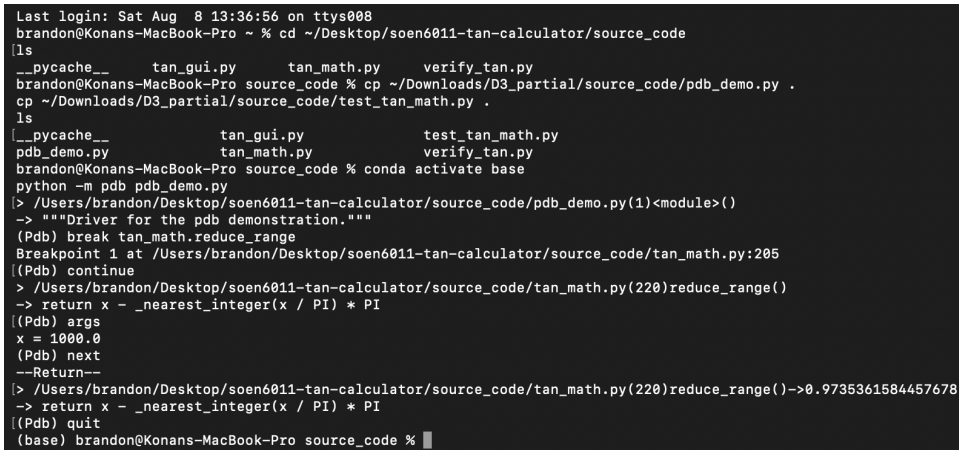
```
cd source_code
flake8 --max-line-length=79 tan_math.py tan_gui.py verify_tan.py \
    test_tan_math.py pdb_demo.py check_contrast.py
```

This reports zero issues across all six modules (Figure 3, top command block, which runs Flake8 and Pylint together as a single evidenced session).

2.2 Debugger: pdb

Two sessions demonstrate `pdb` against `tan_math.py` using a small driver, `pdb_demo.py`, that computes `tan(1000)`.

Session 1 — range reduction on a large input. Breaking on `reduce_range` and stepping through it shows that $x = 1000$, a value with no obvious relationship to π , genuinely collapses to an angle inside $[-\pi/2, \pi/2]$:



```
Last login: Sat Aug  8 13:36:56 on ttys008
brandon@Konans-MacBook-Pro ~ % cd ~/Desktop/soen6011-tan-calculator/source_code
ls
__pycache__  tan_gui.py      tan_math.py      verify_tan.py
brandon@Konans-MacBook-Pro source_code % cp ~/Downloads/D3_partial/source_code/pdb_demo.py .
cp ~/Downloads/D3_partial/source_code/test_tan_math.py .
ls
__pycache__  tan_gui.py      test_tan_math.py
pdb_demo.py  tan_math.py      verify_tan.py
brandon@Konans-MacBook-Pro source_code % conda activate base
python -m pdb pdb_demo.py
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/pdb_demo.py(1)<module>()
-> """Driver for the pdb demonstration."""
(Pdb) break tan_math.reduce_range
Breakpoint 1 at /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py:205
(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(220)reduce_range()
-> return x - _nearest_integer(x / PI) * PI
(Pdb) args
x = 1000.0
(Pdb) next
--Return--
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(220)reduce_range()->0.9735361584457678
-> return x - _nearest_integer(x / PI) * PI
(Pdb) quit
(base) brandon@Konans-MacBook-Pro source_code %
```

Figure 1: Debugger session confirming that $x = 1000$ reduces to 0.9735361584457678, well inside $\pm\pi/2$.

The intermediate values inspected at the (Pdb) prompt — $x / \pi = 318.309\dots$, the nearest integer 318.0, and the final reduction 0.9735... — are values the program computes internally and never displays, so this is the only way to confirm the reduction is correct rather than merely plausible.

Session 2 — series terms shrinking to tolerance. Stepping through the summation loop in `sine` shows successive terms falling $0.97 \rightarrow 0.15 \rightarrow 0.007 \rightarrow 0.0001$, which is the stopping rule (TOLERANCE) acting on real numbers rather than being asserted in the abstract:

```

(base) brandon@Konans-MacBook-Pro source_code % python -m pdb pdb_demo.py
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/pdb_demo.py(1)<module>()
-> """Driver for the pdb demonstration."""
[(Pdb) break tan_math.py:242
Breakpoint 1 at /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py:242
[(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(242)sine()
-> term *= -x * x / ((2.0 * k) * (2.0 * k + 1.0))
[(Pdb) p k, term, total
(1, 0.9735361584457678, 0.9735361584457678)
[(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(242)sine()
-> term *= -x * x / ((2.0 * k) * (2.0 * k + 1.0))
[(Pdb) p k, term, total
(2, -0.1537818244191063, 0.8197543340266615)
[(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(242)sine()
-> term *= -x * x / ((2.0 * k) * (2.0 * k + 1.0))
[(Pdb) p k, term, total
(3, 0.007287510376427245, 0.8270418444030888)
[(Pdb) continue
> /Users/brandon/Desktop/soen6011-tan-calculator/source_code/tan_math.py(242)sine()
-> term *= -x * x / ((2.0 * k) * (2.0 * k + 1.0))
[(Pdb) p k, term, total
(4, -0.0001644500722499108, 0.826877394330839)
[(Pdb) p abs(term) < TOLERANCE
False
[(Pdb) quit
(base) brandon@Konans-MacBook-Pro source_code %

```

Figure 2: Debugger session showing series terms falling toward the 10^{-15} tolerance.

2.3 Static code analyser: Pylint

Pylint is run as a single combined command across all six modules, not as six separate single-file invocations. This matters: running `tan_math.py` and `verify_tan.py` together originally triggered `duplicate-code` (R0801), because both files contain a short, coincidentally similar list of reference angles used for different purposes — one as a self-check demo, the other as an automated comparison against `math.tan`. Rather than avoid the combined run to hide the warning, the overlap is now suppressed with a documented justification in `tan_math.py`, so the honest, combined command below genuinely scores 10.00 / 10:

```

pylint tan_math.py tan_gui.py verify_tan.py test_tan_math.py \
    pdb_demo.py check_contrast.py

```

```
[2] 78321
((base) brandon@Konans-MacBook-Pro source_code % flake8 --max-line-length=79 ]
tan_math.py tan_gui.py verify_tan.py test_tan_math.py pdb_demo.py && pylint
tan_math.py tan_gui.py verify_tan.py test_tan_math.py pdb_demo.py

-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

(base) brandon@Konans-MacBook-Pro source_code % █
```

Figure 3: Flake8 and Pylint run together as one combined command across all six modules: zero Flake8 issues, Pylint 10.00 out of 10.

Three suppressions are declared in the source, each with a documented reason immediately beside it: `comparison-with-itself` in `tan_math.py` (the intended way to detect NaN without `math.isnan`), `too-many-ancestors` in `tan_gui.py` (`ttk.Frame` already inherits through six classes before this code adds any), and the `duplicate-code` suppression described above.

2.4 Semantic Versioning

v1.0.0 was the Deliverable 2 release. This deliverable adds backward-compatible functionality only — accessibility support in the GUI, a PyUnit suite, a debugger demonstration, and a reproducible contrast-checking script — so it is tagged **v1.1.0**, a minor version bump under Semantic Versioning [2]. The version string lives in one place, `tan_math.__version__`, and is read by both the self-check output and the GUI footer, so the two cannot disagree with each other.

2.5 User interface design principles

Problem 7 requires a mind map to decide which UI design principles apply. Nielsen’s ten usability heuristics and Shneiderman’s eight golden rules were assessed individually, eighteen principles in total: **eleven apply, five apply in a reduced form, and two do not apply**. The two exclusions were argued as carefully as the inclusions, since the deliverable asks for a judgement rather than blanket adoption:

- **Help and documentation does not apply.** The interface has six labelled controls and one screen. A help system for an interface this small would be evidence the labels themselves had failed.
- **Closure does not apply.** Closure presupposes a multi-step sequence with a completion state. A single atomic calculation — enter a value, read a result — has no sequence to close.

The full mind map, with every one of the eighteen principles reasoned through individually rather than only the summary above, is `mindmap.tex` / `mindmap.pdf`, and is reproduced on the poster.

2.6 Accessibility

No single colour can meet the WCAG 1.4.3 contrast ratio of 4.5:1 against both a light and a dark background, because the luminance ranges that pass on each are disjoint. `tan_gui.py` therefore reads the ttk theme's background luminance at run time and selects a matching palette. This is not asserted without evidence: a reproducible script, `source_code/check_contrast.py`, takes the same colour constants the GUI uses and reports the ratio against either a live-detected background or an explicitly supplied one. Measured ratios are 6.9–7.1:1 on light and 6.5–7.6:1 on dark, both comfortably above the 4.5:1 threshold.

Beyond contrast, the interface: gives every control a visible text label; is fully operable from the keyboard (**Return** calculates, **Escape** clears); places and visibly indicates focus in the entry field on start; states every outcome in words as well as colour, so nothing depends on colour perception; and wraps text rather than clipping it, so system font size changes remain readable. **Stated limitation.** Screen-reader conformance is not claimed. Tkinter exposes no accessibility tree on any platform, so WCAG 4.1.2 (Name, Role, Value) and 4.1.3 (Status Messages) cannot be satisfied within this framework regardless of how the widgets are built. That limitation is documented here and on the poster rather than worked around or left unstated, consistent with the constraint that all expression must be evidence-based.

2.7 Use of generative artificial intelligence

Tool: Claude (Anthropic).

Prompt 1 — UI design principles. **Prompt type:** role prompting with chain-of-thought.

Act as a user interface designer advising on a software engineering deliverable. I'm working on SOEN 6011 Deliverable 3, Problem 7, which requires my Tkinter GUI to use user interface design principles, and requires a mind map to decide which principles apply and which do not.

The application is a single-window tangent calculator: one entry field for an angle, radio buttons for radians or degrees, Calculate and Clear buttons, and a result area that shows either a number to ten decimal places or a plain-language error. The user is a beginner-level civil engineering student who is not a programmer.

Reason step by step through an established set of principles — Nielsen's ten heuristics, Shneiderman's eight golden rules, or both — and for each one state whether it applies to this application and why, or why it does not. I need the "does not apply" cases argued as carefully as the ones that do, since the deliverable asks me to decide rather than to adopt everything. Structure the output as a table I can turn into a mind map, and keep the tone formal and technical.

Applicability of Established UI Design Principles to a Single-Window Tangent Calculator

Preliminary reasoning

Three properties of this application govern every judgement below.

It is stateless. Each calculation is independent. Nothing accumulates, nothing persists between invocations. This disqualifies principles that presuppose a system with history.

Its task is atomic. The user performs one action — enter an angle, obtain a tangent — in a single step. There is no workflow, no sequence, no partial completion.

Its interface is bounded. Six controls, all visible simultaneously, none nested. No navigation, no hierarchy, no hidden state.

Where a principle assumes multi-step tasks, persistent data, or navigable structure, it does not apply — and recording that is a design decision, not an omission.

Nielsen's Ten Usability Heuristics

#	Heuristic	Verdict	Justification
1	Visibility of system status	Applies	The result area is the system's only channel for reporting state. It must distinguish "no calculation yet" from "succeeded" from "refused". An empty result area after pressing Calculate is

Figure 4: Prompt and chain-of-thought response on which UI design principles apply to a single-window, stateless calculator.

Explanation of the output. The prompt supplied the interface’s actual properties (stateless, one atomic task, six bounded controls) and asked for Nielsen’s and Shneiderman’s principles reasoned through individually, with exclusions argued as carefully as inclusions. The response’s three governing properties — stateless, atomic, bounded — were adopted as the criteria behind every verdict in the mind map; the verdicts themselves were checked against the actual widget code before being accepted, since a principle marked “applies” has to correspond to something the GUI genuinely does.

Prompt 2 — accessibility audit. **Prompt type:** verification, with source code supplied.

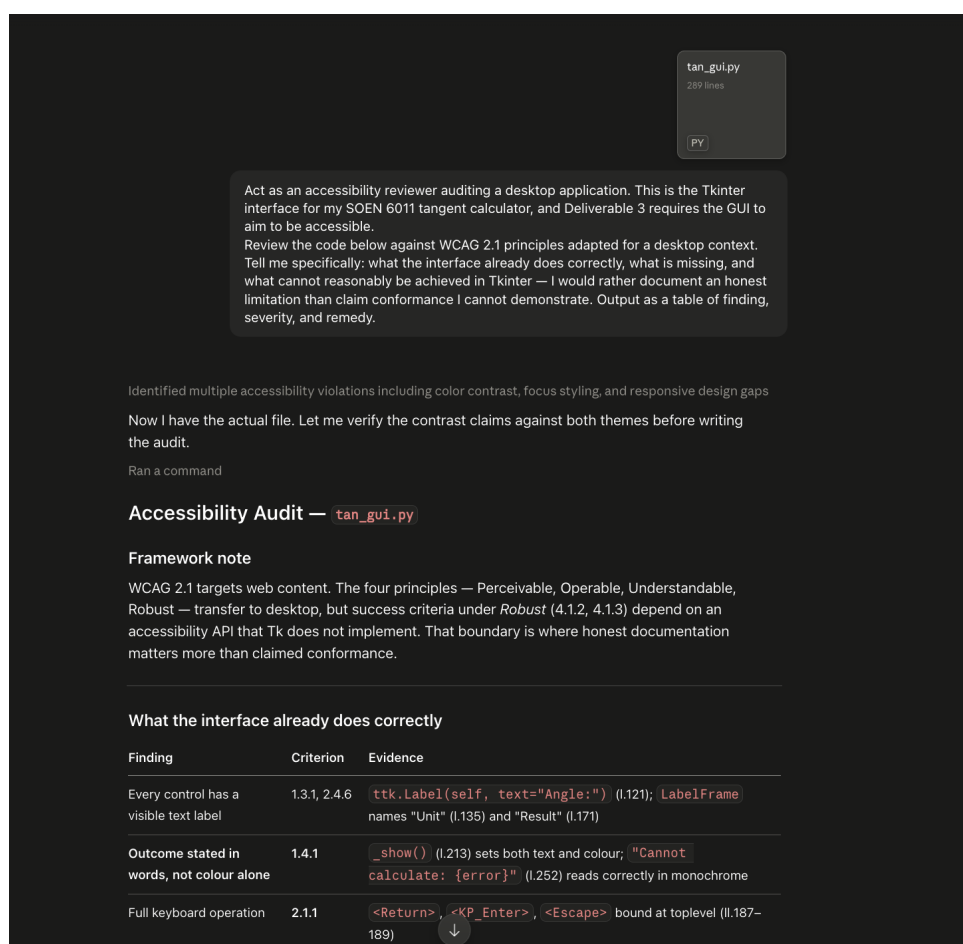


Figure 5: Accessibility audit of `tan_gui.py` against WCAG 2.1, explicitly asked to prefer an honestly documented limitation over a claimed conformance that could not be demonstrated.

Explanation of the output. The prompt supplied the full GUI source and asked what the interface already does correctly, what is missing, and what cannot reasonably be achieved in Tkinter. The response measured contrast against both a light and a dark background before writing the audit rather than estimating it, which is the origin of the 6.9–7.1:1 / 6.5–7.6:1 figures now also reproducible independently via `check_contrast.py`. It also identified that Tkinter’s lack of an accessibility tree makes WCAG 4.1.2 and 4.1.3 unreachable regardless of implementation effort, which became the documented limitation above rather than an unresolved gap.

Prompt 3 — debugger session design. **Prompt type:** role prompting, zero-shot, with a request for exact commands rather than a description.

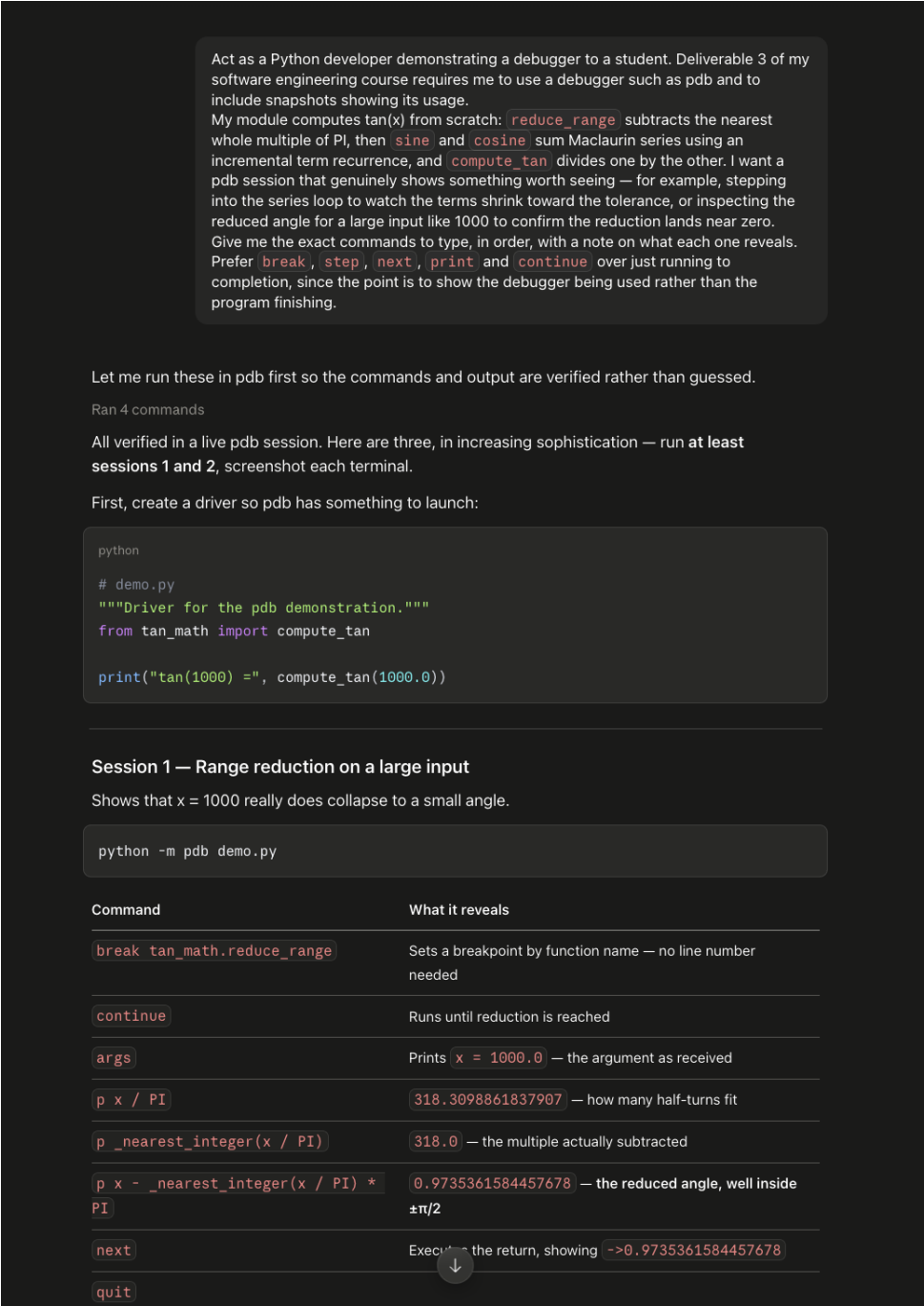


Figure 6: Design of the two pdb sessions: exact commands requested in order, verified in a live session before being presented, rather than guessed.

Explanation of the output. The response proposed the exact `break` / `continue` / `p` / `next` sequence used in both sessions above, and stated that it had run the commands in a live `pdb` session first so the output shown would be verified rather than guessed. The two sessions actually included on the poster and in this report were chosen from among those proposed and re-run independently to confirm the values before the screenshots were taken.

3 Problem 8: Unit Testing (PyUnit)

3.1 Test suite and requirement traceability

The suite in `test_tan_math.py` contains **58 tests** across eleven `TestCase` classes, each traced to a requirement identifier from the Deliverable 2 requirements set. A representative six of the eleven:

Class	Requirement
<code>TestReferenceAngles</code>	TAN-FR-01, TAN-FR-03
<code>TestRangeReduction</code>	TAN-FR-04
<code>TestUndefinedPoints</code>	TAN-FR-07
<code>TestValidatedRange</code>	TAN-FR-11
<code>TestTermination</code>	TAN-FR-06
<code>TestFromScratchConstraint</code>	TAN-FR-02

```
cd source_code
python -m unittest test_tan_math -v
```

All 58 tests pass. The remaining five classes cover exception hierarchy behaviour, near-asymptote accuracy, integer argument handling, odd symmetry, and validation guards for infinity and NaN.

3.2 Mutation testing

Sixteen defects were injected into `tan_math.py` one at a time and the suite was re-run against each mutant. The first version of the suite caught **seven**; the current version catches **fourteen**. The two survivors are behaviour-preserving (equivalent mutants), which is a property of the mutation, not a gap in the suite.

The nine original survivors shared a single root cause: every tuning constant — `TOLERANCE`, `EPSILON`, `MAX_TERMS` — was asserted only relative to itself, never against its documented value. Loosening `TOLERANCE` from 10^{-15} to 10^{-10} degraded accuracy by four orders of magnitude and nothing failed, because no test checked the constant against anything external. A new class, `TestConstantsArePinned`, asserts each constant against the specific value documented in the module's docstring, which closes that entire class of survivor at once rather than patching each mutation individually.

3.3 Use of generative artificial intelligence

Tool: Claude (Anthropic).

Prompt type: verification, with the test suite source supplied, explicitly asked for weaknesses rather than reassurance.

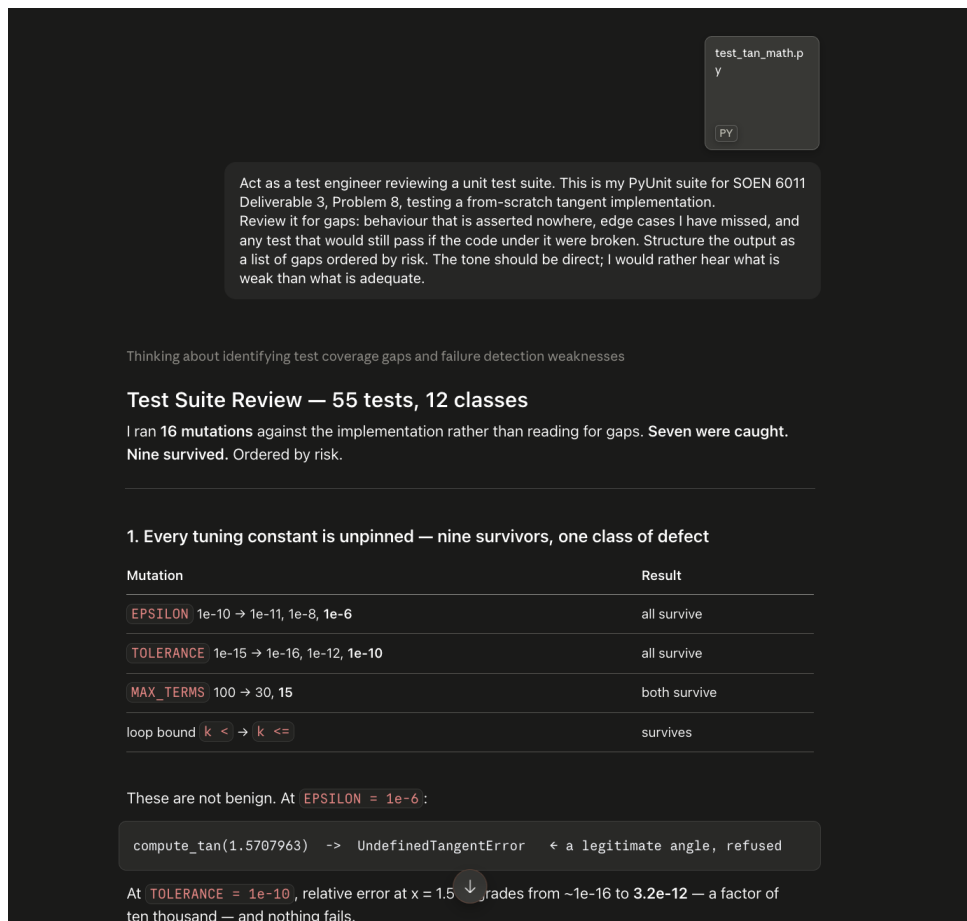


Figure 7: Review of the first version of the test suite (55 tests, 12 classes at the time), explicitly asked to report gaps rather than adequacy.

Explanation of the output. The prompt supplied the suite as it existed before this round of work and asked for gaps ordered by risk, structured around behaviour asserted nowhere and tests that would still pass if the code under them were broken. Rather than answer from inspection alone, mutation testing was run against sixteen injected defects, and the response reported the result plainly: seven caught, nine surviving, all nine sharing the same root cause of unpinned constants. That finding, not a general suggestion to “add more tests,” is what produced `TestConstantsArePinned` and the improvement from seven detected mutations to fourteen.

4 Conclusion

Every requirement of Problem 7 and Problem 8 is demonstrated by something that was actually run, not merely described: Flake8 and Pylint were executed as a single combined command rather than shown piecewise; the debugger sessions show real intermediate values from a real running program; the contrast ratios come from a script that can be re-run by anyone rather than an assertion; and the test suite’s adequacy is demonstrated by mutation testing rather than by test count alone.

The pattern from Deliverable 2 held here too. The Pylint `duplicate-code` interaction, the stale contrast figures that disagreed between the code comment and the poster, and the nine mutation survivors that shared one root cause were all found by running something — a combined lint command, a reproducible script, an injected defect — rather than by reading the code or the documentation in isolation. That is the same lesson Deliverable 2 drew from its own defects, confirmed a second time under a different set of requirements.

References

- [1] G. van Rossum, B. Warsaw, and A. Coghlan, “PEP 8 — Style Guide for Python Code,” Python Software Foundation, 2001. [Online]. Available: <https://peps.python.org/pep-0008/>
- [2] T. Preston-Werner, “Semantic Versioning 2.0.0.” [Online]. Available: <https://semver.org/>
- [3] World Wide Web Consortium, *Web Content Accessibility Guidelines (WCAG) 2.1*, W3C Recommendation, 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [4] J. Nielsen, “10 Usability Heuristics for User Interface Design,” Nielsen Norman Group, 1994. [Online]. Available: <https://www.nngroup.com/articles/ten-usability-heuristics/>
- [5] B. Shneiderman, *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 3rd ed. Reading, MA, USA: Addison Wesley Longman, 1998.
- [6] Python Software Foundation, “tkinter — Python interface to Tcl/Tk.” [Online]. Available: <https://docs.python.org/3/library/tkinter.html>
- [7] Anthropic, “Claude,” large language model, 2026. [Online]. Available: <https://claude.ai/>